

Machine Learning com R e Python

Ana Carolina Gontijo, André Vinícius, Filipe Silva, Gabriel César e Gabriel Victor.

Introdução

Base de dados escolhida: Heart Attack Data Set / Heart Disease Dataset

Fundamentação teórica

- **Naive Bayes:**

É um algoritmo probabilístico baseado no Teorema de Bayes.

Assume que as variáveis independentes não têm correlação entre si.

Modelo rápido computacionalmente e costuma entregar bons resultados como uma *baseline* em sistemas de saúde.

- **Random Forest:** É um método *ensemble* (conjunto) que treina várias Árvores de Decisão simultaneamente. Ele cria múltiplas árvores com subconjuntos aleatórios dos dados e faz uma votação entre elas para a predição final. Isso corrige o problema de *overfitting* das árvores simples e aumenta a robustez, ao custo de reduzir a interpretabilidade (atuando mais como uma "caixa preta").

- **Árvore de Decisão:** Funciona criando regras de separação nos dados, formando uma estrutura de fluxograma. O algoritmo escolhe as variáveis que melhor dividem as classes usando critérios como o Índice Gini ou Entropia. É um modelo excelente para a área médica devido à sua alta interpretabilidade, mas possui a desvantagem de sofrer facilmente com *overfitting* (decorar os dados de treino).

Metodologia

A base de dados utilizada foi extraída do Kaggle (Heart Attack Data Set / Heart Disease Dataset) e contém variáveis clínicas relevantes como idade, pressão arterial, colesterol e ritmo cardíaco.

- **Divisão dos dados:** 80% para treino e 20% para teste em ambas as linguagens.
- **Amostragem:** Amostragem estratificada foi aplicada para garantir que a proporção de pacientes doentes e saudáveis fosse mantida igual nas duas partições.
- **Padronização:** Os dados numéricos foram escalonados (utilizando StandardScaler em Python e preProcess em R), garantindo que variáveis com escalas muito diferentes não enviesassem algoritmos sensíveis a distâncias.
- **Reprodutibilidade:** Foi fixada a mesma semente aleatória (seed = 42) tanto em Python quanto em R para garantir uma comparação justa e reproduzível.

Implementação

Python: Utilizamos o ecossistema do scikit-learn. Os modelos foram instanciados utilizando GaussianNB, DecisionTreeClassifier e RandomForestClassifier. Realizamos o ajuste de hiperparâmetros (como a profundidade máxima da árvore - max_depth, e o número de estimadores no Random Forest - n_estimators) utilizando a função GridSearchCV com validação cruzada (cv=5), otimizando a configuração do modelo nos dados de treino.

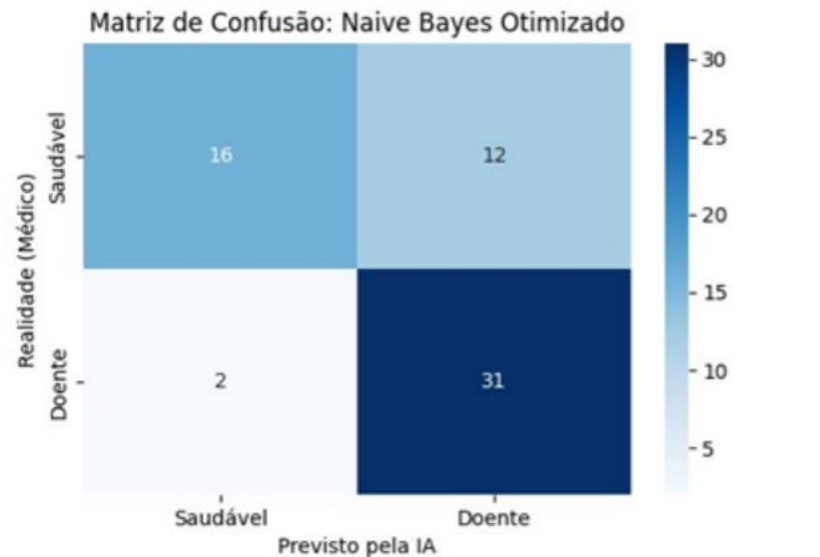
R: O fluxo de trabalho foi centralizado no pacote caret, integrando o pacote rpart para a indução da Árvore de Decisão, randomForest para o ensemble, e e1071 para o Naive Bayes. A função trainControl foi utilizada para gerenciar a validação cruzada e o grid de hiperparâmetros nativo do caret foi explorado para encontrar os melhores ajustes.

Resultados

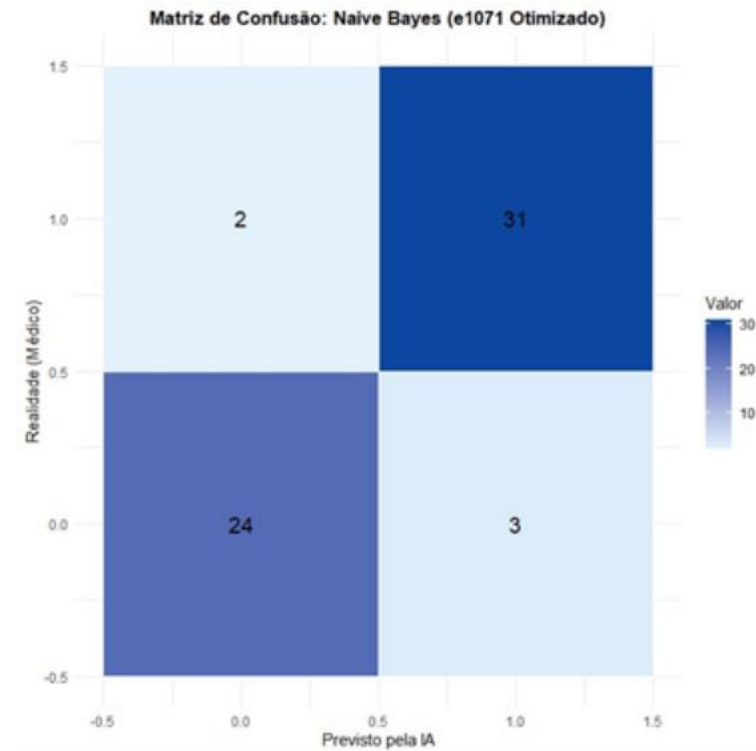
Modelo / Linguagem	Acurácia	Precisão	Revocação	F1-Score
Naive Bayes (Python)	77.05%	80.00%	76.00%	76.00%
Naive Bayes (R)	91.67%	92.30%	88.88%	90.56%
Árvore de Decisão (Python)	78.69%	75.00%	90.91%	82.19%
Árvore de Decisão (R)	75.00%	72.73%	80.00%	76.19%
Random Forest (Python)	90.16%	86.49%	96.97%	91.43%
Random Forest (R)	85.00%	90.90%	74.10%	81.60%

Matrizes de confusão- Naive Bayes

Python

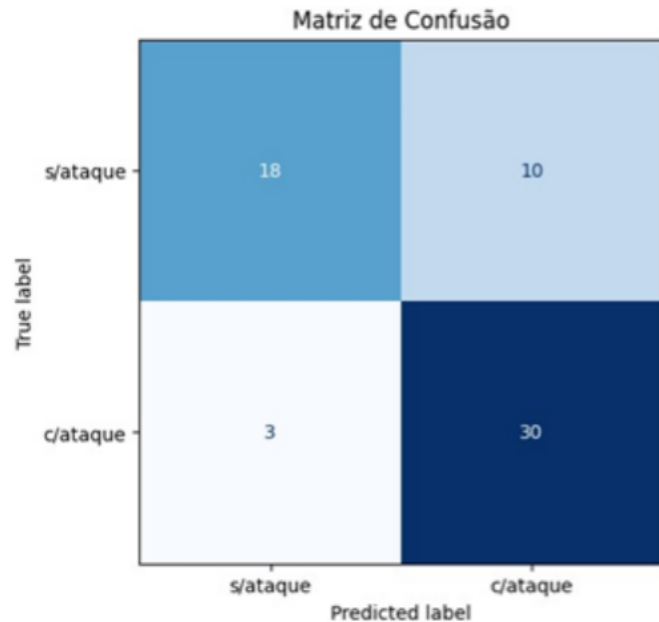


R

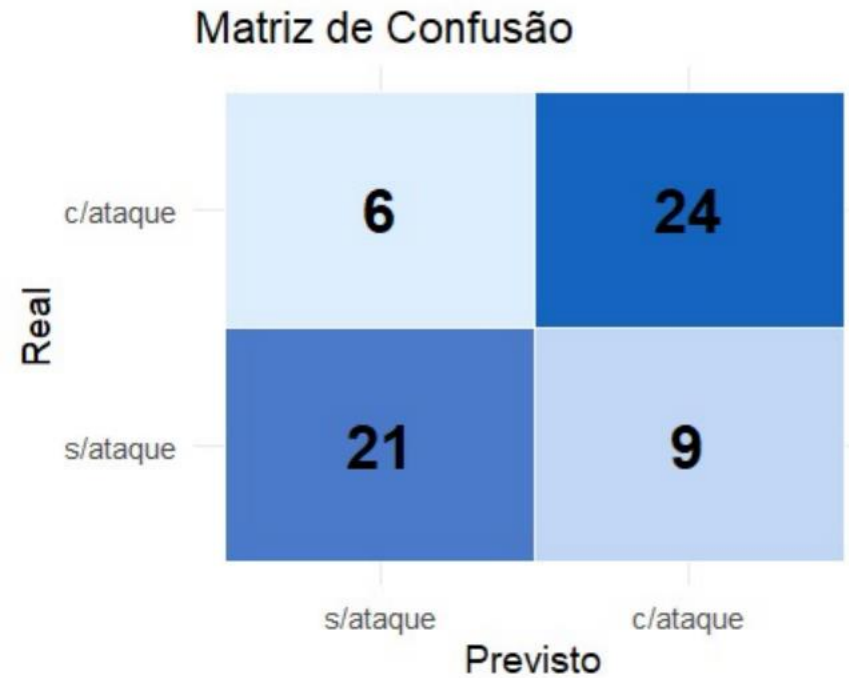


Matrizes de cofusão- Árvore de decisão

Python

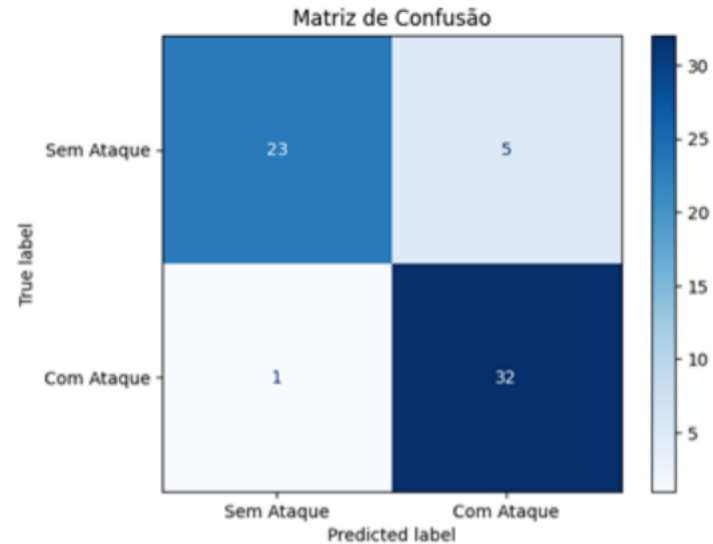


R



Matrizes de confusão- Random Forest

Python



R

Matriz de Confusão



Discussão

- Os resultados realçam porque a avaliação isolada da acurácia é ineficiente
- A árvore de decisão demonstrou forte tendência a maximizar o Recall. O trade-off disso foi uma redução de precisão com mais alarmes falsos
- O random forest oferece um balanço mais harmônico entre precisão e recall ao custo de maiores processamento computacional e perda de interpretabilidade
- Em relação ao ecossistema, o scikit-learn permitiu um código altamente modular
- R exigiu um manejo mais cuidadoso e mostrou-se mais poderoso com o pacote caret
- divergência inicial entre os resultados do Naive Bayes em Python (77.05% de acurácia) e em R (91.67%)
- A abordagem por Kernel permitiu ao modelo em R moldar de forma não-paramétrica as distribuições reais das variáveis clínicas de saúde resultando em uma capacidade preditiva superior para este algoritmo específico

Conclusão

ORandomForest sobressai como o modelo preditivo mais robusto e estável quando focado na capacidade bruta de acerto global.

No entanto, em um ambiente hospitalar onde a transparência da decisão médica é crítica, a Árvore de Decisão desponta como a opção mais adequada por apresentar claramente seus nós de decisão, mesmo sacrificando alguns pontos percentuais de desempenho.

A comparação entre R e Python revelou desempenhos preditivos muito semelhantes — comprovando que, com a mesma semente aleatória e partição de dados, o diferencial competitivo de um projeto de Machine Learning está muito mais atrelado a um pré-processamento minucioso do que à linguagem adotada.

